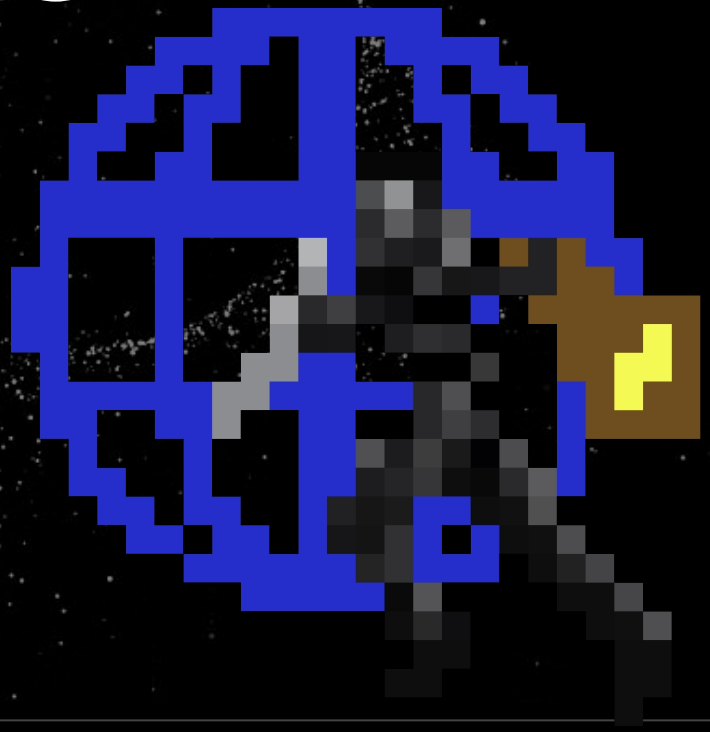
WebRogue

Jacob Martinez



A language for
Web Games

Check out the Companion Website!

Why WebRogue? What is your Story?

Remember Pygame in 1010?

Because I didn't like it at all, I wanted to create a DSL that was lightweight and easy to create games on.

I HATE IT!

So I decided create a language that focuses on readability, ease of use, and small game dev features for beginners.

Basic Description (Just Skim This)

WebRogue is a statically-typed, imperative programming language designed specifically for scripting browser-based roguelike games. It compiles to readable JavaScript and emphasizes compile-time safety checks to catch common game scripting errors. The language is intentionally compact, making it suitable for educational compiler projects while providing enough structure for practical game development.

Classic JS Type Aspects

- `number`: Floating-point numeric values
- `string`: Text strings with standard escape sequences
- `boolean`: True/false values
- `void`: Used only for function return types

```
let hp: number = 100;
let name = "Hero";  // Type inferred from initializer
```

Functions are first-class with typed parameters and return values:

```
function calculateDamage(base: number, multiplier: number)
 -> number {
  return base * multiplier;
}
```

Static Analysis

The compiler performs comprehensive static checks including:

- Variable declaration before use
- Type consistency in assignments and operations
- Function call argument matching
- Control flow validity (break only in loops, return only in functions)
- Game-specific constraints (unique object/state names, valid state transitions)

Compilation Target

WebRogue programs compile to JavaScript, generating both runnable game scripts and plain JavaScript objects suitable for browser execution. The generated code maintains readability for debugging and integration with larger web applications.

Themed Example

All of our language tries to be more clear for beginner users.

- Entities and other objects are come with built in labels for name, hp, status, etc.
  These act as your typical classes but are more narrowed for specific use.
- States represent screens, dialogue, and choice options.

examples > tinyDungeon.wr

```
object Player {
  name: string = "Mira";
  hp: number = 12;
  alive: boolean = true;
}

object Slime {
  name: string = "Slime";
  hp: number = 4;
  alive: boolean = true;
}

state Start {
  title: "Dungeon Start";
  description: "A half-burned torch marks the safe edge of the dungeon.";
  contains: [Player];
}

state Entry {
  title: "Mossy Entry";
  description: "Cold stairs descend into the first pocket of dungeon air.";
  contains: [Player, Slime];
}

function damage(hp: number, amount: number) -> number {
  return hp - amount;
}

let playerHp: number = 12;
if playerHp > 0 {
  print "ready";
} else {
  print "fallen";
}

while playerHp > 9 {
  playerHp = damage(playerHp, 1);
  if playerHp == 10 {
    break;
  }
}

_jump(Entry);
print "entered Entry";
print playerHp;
```

Unique Ohm Quirks

```
JumpStmt     = jump "(" id ")" ";"
ChoiceStmt   = choice "(" ChoiceInput ChoiceOptions? ")" "{" ChoiceArm+ "}"
ChoiceInput  = keyboard "." id                                  --keyboard
             | id                                               --id
ChoiceOptions = "," IdList
ChoiceArm    = option id Block                                  --block
             | option id ":" Block                              --colon
ExpStmt      = Exp ";"

FunctionDecl = function id "(" Params? ")" ReturnType? Block
Params       = NonemptyListOf<Param, ",">
Param        = id ":" Type
ReturnType   = "->" Type

ObjectDecl   = object id "{" ObjectField* "}"
ObjectField  = id ":" Type "=" Exp ";"

StateDecl    = state id "{" StateField* "}"
StateField   = title ":" string ";"                             --title
             | description ":" string ";"                       --description
             | contains ":" "[" IdList? "]" ";"                 --contains
             | DialogueField                                    --dialogue
DialogueField = dialogue "(" ChoiceInput "," DialogueStrings ")" ";"?
DialogueStrings = NonemptyListOf<string, ",">
IdList       = NonemptyListOf<id, ",">
```

- Choice -
- State -
- Jump -
- Dialogue -

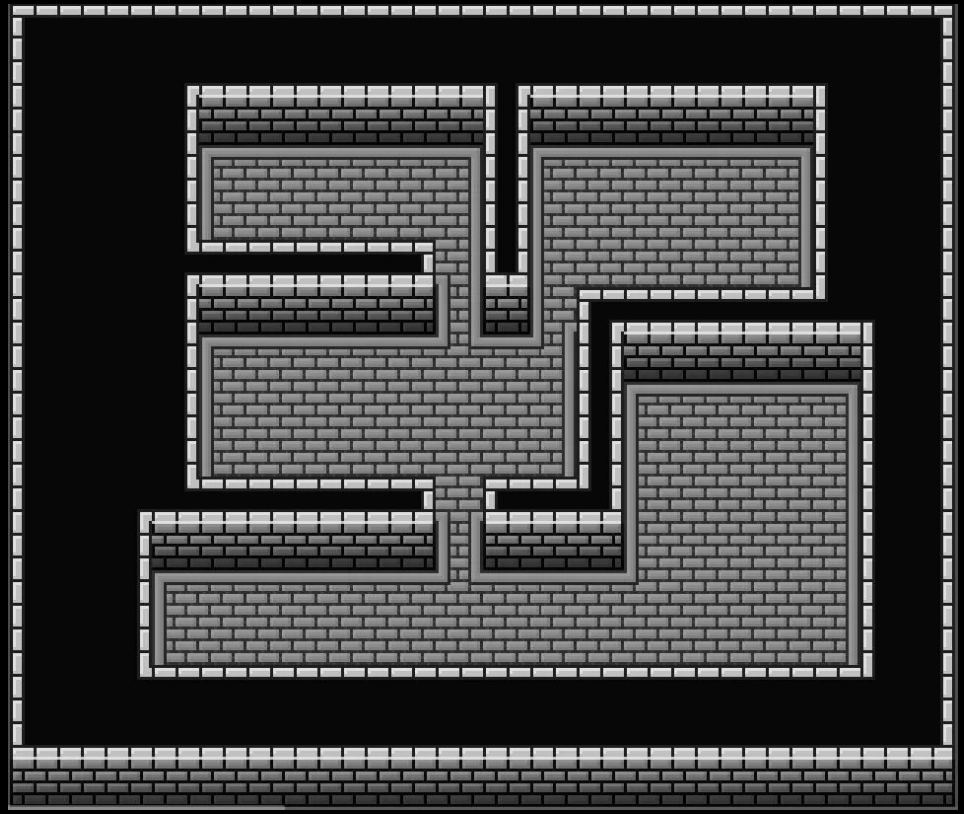
I wanted to emulate some small game engine-like features. So I implemented a few features / shortcuts to commonly used aspects in games.
Choice to accept player input.
State to manage player state, _jump() method. to move between those states.
And a small dialogue system.

Future Implementation

Something I couldn't implement feasibly in time is a tilemap render feature.

I was in the process of implementing a tilemap shortcut within state to render a map.

```
Tilemap - {
# # # # #
# .  .  . #   - creates a 3x3 room outlined
# .  p . #      by walls with the player in
# .  .  . #     the middle.
# # # # #
}
```



Bonus Question Time?

How do you think WebRouge expects file and variables to be **named**?

In **snakeCase** of course!

Thank you